

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE MATEMÁTICA E ESTATÍSTICA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Rafael Vieira de Carvalho

Divisão de nós em uma max-tree

Atualização dinâmica de max-trees sob transformações anti-extensivas locais

Orientador: Prof. Dr. Ronaldo F. Hashimoto
Coorientador: Prof. Dr. Wonder A. L. Alves

**São Paulo
2026**

1 Resumo

2 Introdução e Justificativas

- Motivação para o que queremos estudar
 - Explicar o que é morfologia matemática [1]
 - Histórico de EDs utilizadas no passado para morfologia matemática
 - Explicar sobre component trees
 - Explicar hipóteses necessárias para construção de component tree [12, 13, 14]
 - Afunilar a ED que será explorada (max tree)
 - Explicar para que component tree é utilizada [23, 24]
 - Explicar por que é bastante utilizada [34, 26, 10, 27]
- O que é o problema da reconstrução
- Qual é o estado da arte "merge"
- Qual a lacuna que pretende-se pesquisar "split"
- explicar como está dividido esse documento

A morfologia matemática é uma subárea do processamento de imagens e da visão computacional que tem como objetivo estudar e desenvolver transformações aplicadas a imagens, tendo como principal arcabouço teórico a teoria dos conjuntos [3]. Conforme sugere o próprio termo *morfologia*, essa área busca analisar e extrair informações relacionadas às características geométricas dos objetos presentes nas imagens, como forma, tamanho e estrutura.

Nessa busca pelo desenvolvimento de transformações, o uso de estrutura de dados, para representar e aplicar transformações em imagens, é extremamente necessário. Exemplos de estruturas são: quad-trees [10, 1], binary partition tree [6], watershed tree [2]

A *component tree* [7] é uma das estruturas utilizadas para representar imagens. A ideia fundamental por trás dessa estrutura consiste em representar uma imagem a partir da ordenação de seu conjunto de pixels e de suas componentes conexas. Para isso, é necessário definir o conceito de componente conexa em uma imagem [5, 4] e, de forma mais geral, estabelecer como uma imagem pode ser representada por meio de um grafo [3].

A representação de uma imagem como um grafo é realizada de forma direta, na qual os vértices correspondem aos pixels da imagem, enquanto as arestas são definidas de acordo com um determinado modelo de conectividade [5].

Conforme dito anteriormente, uma das premissas para a construção da component tree é a ordenação do conjunto de pixels. Existem duas formas de ordenar os pixels: de maneira ascendente ou de maneira descendente. Essas duas formas dão origem a duas estruturas de dados duais, a max tree cuja ordenação é ascendente, ou seja, a relação de ordenação dos pixels se dá pelo operador $\leq$ entre os níveis de cinza. E a min tree cuja ordenação é descendente, ou seja, a relação de ordenações dos pixels se dá pelo operador $\geq$ entre os níveis de cinza.

A component tree originalmente foi desenhada para realizar operações conexas com o propósito de resolver tarefas de filtragem e segmentação [8, 9]. Para realizar tais tarefas a operação mais comum utilizada é o corte, que pode consistir na remoção de alguns nós baseado na otimização de algum critério ou na remoção de alguns nós baseado nos atributos contidos nos nós.

3 Objetivos

- Construir arcabouço teórico
- **Objetivo Geral:** Declarar de forma direta o que pretende-se entregar
- **Objetivo Específico:** Mapear os passos concretos necessários para alcançar o objetivo geral (fundamentação matemática, algoritmo, suporte a atributos, demonstrações de corretude/complexidade).

4 Metodologia

Tópicos para a construção de metodologia

- Construção do arcabouço teórico para elaborar o algoritmo
- Demonstrações do arcabouço teórico
- Elaboração do algoritmo

- Demonstração de corretude do algoritmo
- Análise da complexidade computacional do algoritmo
- Implementação do algoritmo
- Elaboração de métricas para o teste do algoritmo
- Teste do algoritmo em algum dataset (a ser definido)
- Teste utilizando o rebuild
- Comparação entre rebuild e split

5 Plano de trabalho e cronograma

Referências

- [1] Gregory M. Hunter and Kenneth Steiglitz. Operations on Images Using Quad Trees. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):145–153, April 1979.
- [2] L. Najman and M. Schmitt. Geodesic saliency of watershed contours and hierarchical segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(12):1163–1173, December 1996.
- [3] Laurent Najman and Jean Cousty. A graph-based mathematical morphology reader. *Pattern Recognition Letters*, 47:3–17, October 2014.
- [4] Benjamin Perret. Inf-structuring Functions: A Unifying Theory of Connections and Connected Operators. *Journal of Mathematical Imaging and Vision*, 51(1):171–194, January 2015.
- [5] Azriel Rosenfeld. Connectivity in Digital Pictures. *Journal of the ACM*, 17(1):146–160, January 1970.
- [6] P. Salembier and L. Garrido. Binary partition tree as an efficient representation for image processing, segmentation, and information retrieval. *IEEE Transactions on Image Processing*, 9(4):561–576, April 2000.

- [7] P. Salembier, A. Oliveras, and L. Garrido. Antiextensive connected operators for image and sequence processing. *IEEE Transactions on Image Processing*, 7(4):555–570, April 1998.
- [8] P. Salembier and J. Serra. Flat zones filtering, connected operators, and filters by reconstruction. *IEEE Transactions on Image Processing*, 4(8):1153–1160, August 1995.
- [9] Philippe Salembier and Michael Wilkinson. Connected operators. *IEEE Signal Processing Magazine*, 26(6):136–157, November 2009.
- [10] S. Tanimoto and T. Pavlidis. A hierarchical data structure for picture processing. *Computer Graphics and Image Processing*, 4(2):104–119, June 1975.